

Todo Management Application

Team Name: Water Tribe

Names: Sulakna Karunaratna, William Haddad, Virgil Santos

Application Features:

User:

- Register a new account
- Login securely
- Logout
- Password encryption
- User authentication using JWT

Todo:

- Create Todo
- View all Todos
- Update Todo
- Delete Todo

Subtask:

- Create Subtask
- View Subtasks
- Update Subtask
- Delete Subtask

Task Breakdown:

1: Our API should be able to process new User registrations.

- As a user, I should be able to create a new Account on the endpoint POST `api/auth/register`. The body will contain a representation of a JSON Account, but will not contain an `accountId`.
- The registration will be successful if and only if the username is not blank, the password is at least 4 characters long, and an Account with that username does not already exist. If all these conditions are met, the response body should contain a JSON of the Account, including its `accountId`. The response status should be 200 OK, which is the default. The new account should be persisted to the database.
- If the registration is not successful due to a duplicate username, the response status should be 409. (Conflict)
- If the registration is not successful for some other reason, the response status should be 400. (Client error)

2: Our API should be able to process User logins.

- As a user, I should be able to verify my login on the endpoint POST `api/auth/login`. The request body will contain a JSON representation of an Account.
- The login will be successful if and only if the username and password provided in the request body JSON match a real account existing on the database. If successful, the response body should contain a JSON of the account in the response body, including its `accountId`. The response status should be 200 OK, which is the default.
- If the login is not successful, the response status should be 401. (Unauthorized)

3: Our API should be able to process the creation of new Todo.

- As a user, I should be able to submit a new post on the endpoint POST `api/todos`. The request body will contain a JSON representation of a todo, which should be persisted to the database, but will not contain a `id`.
- The creation of the todo will be successful if and only if the task is not blank, is not over 255 characters, and `postedBy` refers to a real, existing user. If successful, the response body should contain a JSON of the Todo, including its `id`. The response status should be 200, which is the default. The new Todo should be persisted to the database.
- If the creation of the Todo is not successful, the response status should be 400. (Client error)

4: Our API should be able to retrieve all subtasks from Todo.

- As a user, I should be able to submit a GET request on the endpoint GET `api/todos/{todold}/subtasks`.
- The response body should contain a JSON representation of a list containing all subtasks retrieved from the database. It is expected for the list to simply be empty if there are no subtasks. The response status should always be 200, which is the default.

5: Our API should be able to retrieve a todo by its ID.

- As a user, I should be able to submit a GET request on the endpoint GET `api/todos/{todold}`.
- The response body should contain a JSON representation of the todo identified by the `todold`. It is expected for the response body to simply be empty if there is no such todo. The response status should always be 200, which is the default.

6: Our API should be able to delete a todo identified by a todo ID.

- As a User, I should be able to submit a DELETE request on the endpoint DELETE `api/todos/{todold}`.
- The deletion of an existing todo should remove an existing todo from the database. If the todo existed, the response body should contain the number of rows updated (1). The response status should be 200, which is the default.
- If the todo did not exist, the response status should be 200, but the response body should be empty. This is because the DELETE verb is intended to be idempotent, ie, multiple calls to the DELETE endpoint should respond with the same type of response.

7: Our API should be able to update a todo text identified by a todo ID.

- As a user, I should be able to submit a PATCH request on the endpoint PATCH `api/todos/{todold}`. The request body should contain a new task values to replace the task identified by `todold`. The request body can not be guaranteed to contain any other information.
- The update of a todo should be successful if and only if the todo id already exists and the new task is not blank and is not over 255 characters. If the update is successful, the response body should contain the number of rows updated (1), and the response status should be 200, which is the default. The todo existing on the database should have the updated task.

- If the update of the todo is not successful for any reason, the response status should be 400. (Client error)

8: Our API should be able to delete a subtask identified by a subtask ID.

- As a User, I should be able to submit a DELETE request on the endpoint DELETE `api/subtasks/{subtaskId}`.
- The deletion of an existing todo should remove an existing subtask from the database. If the subtask existed, the response body should contain the number of rows updated (1). The response status should be 200, which is the default.
- If the subtask did not exist, the response status should be 200, but the response body should be empty. This is because the DELETE verb is intended to be idempotent, ie, multiple calls to the DELETE endpoint should respond with the same type of response.

9: Our API should be able to update a subtask text identified by a todo ID.

- As a user, I should be able to submit a PATCH request on the endpoint PATCH `api/subtasks/{subtaskId}`. The request body should contain a new subtask values to replace the task identified by subtaskId. The request body can not be guaranteed to contain any other information.
- The update of a subtask should be successful if and only if the subtask id already exists and the new task is not blank and is not over 255 characters. If the update is successful, the response body should contain the number of rows updated (1), and the response status should be 200, which is the default. The todo existing on the database should have the updated task.
- If the update of the subtask is not successful for any reason, the response status should be 400. (Client error)

10: Our API should be able to process the creation of new Todo.

- As a user, I should be able to submit a new post on the endpoint POST `api/subtaskss`. The request body will contain a JSON representation of a subtask, which should be persisted to the database, but will not contain a id.
- The creation of the subtask will be successful if and only if the subtask is not blank, is not over 255 characters, and postedBy refers to a real, existing user. If successful, the response body should contain a JSON of the subtask, including its id. The response status should be 200, which is the default. The new subtask should be persisted to the database.

- If the creation of the subtask is not successful, the response status should be 400. (Client error)

11: Our API should be able to retrieve all Todos.

- As a user, I should be able to submit a GET request on the endpoint GET api/todos.
- The response body should contain a JSON representation of a list containing all todos retrieved from the database. It is expected for the list to simply be empty if there are no todos. The response status should always be 200, which is the default.

Database tables (Entities):

User Table: ID (Int), Name (string), Email (string), Password(String)

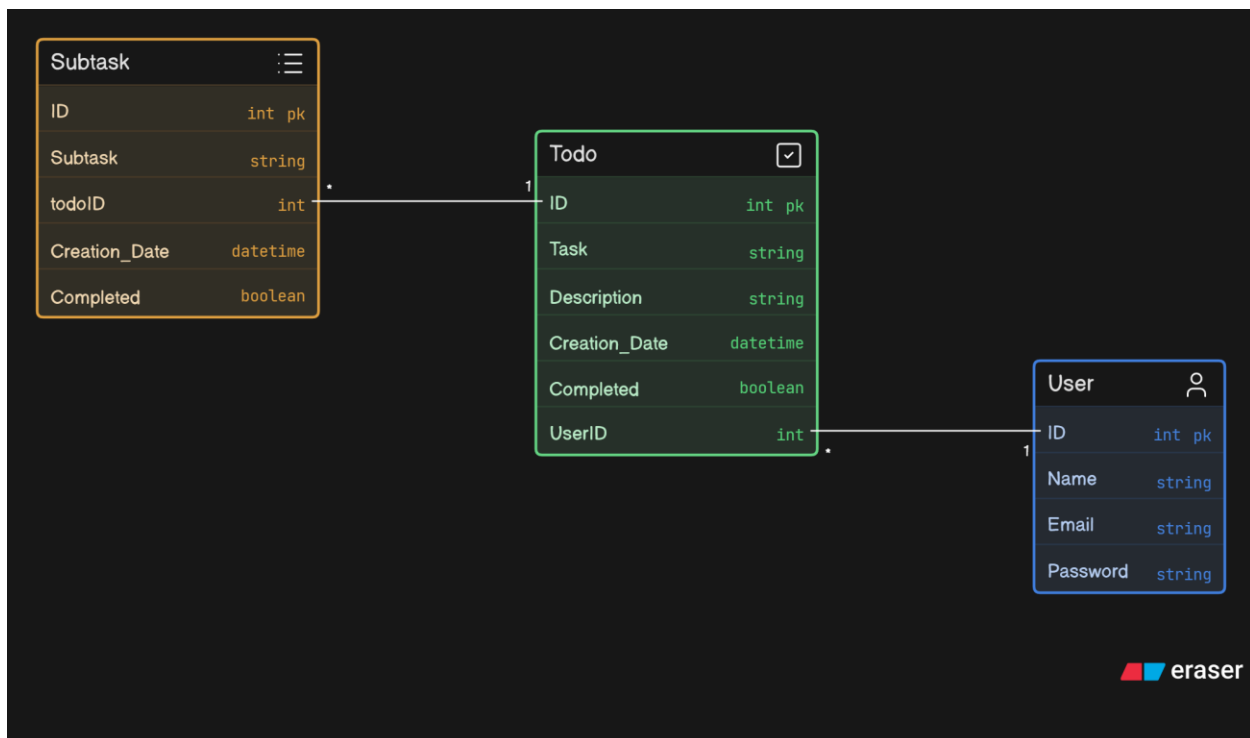
Todo Table: ID (Int), Task(string), Description(string), Creation_Date (DateTime), Completed (Boolean), User ID (Foreign Key)

Subtask Table: ID (Int), Subtask (String), todoID (Foreign Key), Creation_Date (DateTime), Completed (Boolean)

Relations between each Entity:

One User to Many Todos: a user can have many tasks, and each task belongs to only one user

One Main Task to Many Subtasks: a task can have many subtasks, and each subtask derives from exactly one main task.



API Contract: REST API Design

Method	Endpoint	Description
POST	/api/auth/register	Register user
POST	/api/auth/login	Login user
GET	/api/todos	Get all todos
GET	/api/todos/{todold}	Get todo
POST	/api/todos	Create todo
PUT	/api/todos/{todold}	Update todo
DELETE	/api/todos/{todold}	Delete todo
GET	/api/todos/{todold}/subtasks	Get subtasks
POST	/api/todos/{todold}/subtasks	Create subtask
PUT	/api/subtasks/{subtaskId}	Update subtask
DELETE	/api/subtasks/{subtaskId}	Delete subtask

1. Authentication APIs

1.1 Register User

Endpoint: POST /auth/register

Request Body

```
{
  "username": "sula",
  "email": "sula@example.com",
  "password": "Password123"
}
```

Success Response — 201 Created

```
{
  "id": 1,
  "username": "sula",
  "email": "sula@example.com",
  "message": "User registered successfully"
}
```

Error Response — 400 Bad Request

```
{
  "error": "Email already exists"
}
```

1.2 Login User

Endpoint: POST /auth/login (Authenticates a registered user and returns a JWT token)

Request Body

```
{
  "email": "sula@example.com",
  "password": "Password123"
}
```

Success Response — 200 OK

```
{
  "token": "jwt-token-here",
  "tokenType": "Bearer",
  "user": {
    "id": 1,
    "username": "sula",
    "email": "sula@example.com"
  }
}
```

```
}  
}
```

Error Response — 401 Unauthorized

```
{  
  "error": "Invalid email or password"  
}
```

1.3 Logout User

Endpoint: POST /auth/logout (Logs out the user on the frontend by removing the JWT token from local storage or session storage)

Headers

Authorization: Bearer jwt-token-here

Success Response — 200 OK

```
{  
  "message": "User logged out successfully"  
}
```

2. Todo APIs

All Todo endpoints require authentication.

Required Header

Authorization: Bearer jwt-token-here

2.1 Create Todo

Endpoint: POST /todos (Creates a new Todo item for the logged-in user.)

Request Body

```
{  
  "title": "Shopping List",  
  "description": "List for Monday",  
}
```



```
"completed": false
}
```

Success Response — 201 Created

```
{
  "id": 1,
  "title": "Shopping List",
  "description": "List for Monday",
  "completed": false,
  "createdAt": "2026-06-11T10:30:00",
  "updatedAt": "2026-06-11T10:30:00",
  "userId": 1
}
```

2.2 Get All Todos

Endpoint: GET /todos (Returns all Todo items belonging to the logged-in user)

Success Response — 200 OK

```
[
  {
    "id": 1,
    "title": "Shopping List",
    "description": "List for Monday",
    "completed": false,
    "createdAt": "2026-06-11T10:30:00",
    "updatedAt": "2026-06-11T10:30:00",
    "subtasks": [
      {
        "id": 1,
        "title": "Buy Milk",
        "completed": true
      },
      {
        "id": 2,
        "title": "Buy Tea",
        "completed": false
      }
    ]
  }
]
```

```
]
}
]
```

2.3 Get Todo By ID

Endpoint: GET /todos/{todoid} (Returns one Todo item by ID)

Success Response — 200 OK

```
{
  "id": 1,
  "title": "Shopping List",
  "description": "List for Monday",
  "completed": false,
  "createdAt": "2026-06-11T10:30:00",
  "updatedAt": "2026-06-11T10:30:00",
  "subtasks": [
    {
      "id": 1,
      "title": "Buy Milk",
      "completed": true
    }
  ]
}
```

Error Response — 404 Not Found

```
{
  "error": "Todo not found"
}
```

2.4 Update Todo

Endpoint: PUT /todos/{todoid} (Updates an existing Todo item)

Request Body

```
{
  "title": "Shopping List",
  "description": "List for Tuesday",
}
```

```
"completed": true
}
```

Success Response — 200 OK

```
{
  "id": 1,
  "title": "Shopping List",
  "description": "List For Tuesday",
  "completed": true,
  "createdAt": "2026-06-11T10:30:00",
  "updatedAt": "2026-06-11T12:00:00"
}
```

2.5 Delete Todo

Endpoint: DELETE /todos/{id} (Deletes a Todo item and all related subtasks)

Success Response — 200 OK

```
{
  "message": "Todo deleted successfully"
}
```

Error Response — 404 Not Found

```
{
  "error": "Todo not found"
}
```

3. Subtask APIs

All Subtask endpoints require authentication.

Required Header

Authorization: Bearer jwt-token-here

3.1 Create Subtask

Endpoint: POST /todos/{todold}/subtasks (Creates a new subtask under a selected Todo item)

Request Body

```
{  
  "title": "Buy Milk",  
  "completed": false  
}
```

Success Response — 201 Created

```
{  
  "id": 1,  
  "title": "Buy Milk",  
  "completed": false,  
  "todold": 1  
}
```

Error Response — 404 Not Found

```
{  
  "error": "Todo not found"  
}
```

3.2 Get Subtasks By Todo ID

Endpoint: GET /todos/{todold}/subtasks (Returns all subtasks under a specific Todo item.)

Success Response — 200 OK

```
[  
  {  
    "id": 1,  
    "title": "Create login page",  
    "completed": true,  
    "todold": 1  
  },  
  {  
    "id": 2,  
    "title": "Create todo dashboard",  
    "completed": false,  
    "todold": 1  
  }  
]
```

```
"completed": false,  
"todold": 1  
}  
]
```

3.3 Update Subtask

Endpoint: PUT /subtasks/{subtaskId} (Updates an existing subtask)

Request Body

```
{  
  "title": "Buy Choccolate",  
  "completed": true  
}
```

Success Response — 200 OK

```
{  
  "id": 1,  
  "title": "Buy Chocolate",  
  "completed": true,  
  "todold": 1  
}
```

Error Response — 404 Not Found

```
{  
  "error": "Subtask not found"  
}
```

3.4 Delete Subtask

Endpoint: DELETE /subtasks/{subtaskId} (Deletes a subtask without deleting the parent Todo)

Success Response — 200 OK

```
{  
  "message": "Subtask deleted successfully"  
}
```

Error Response — 404 Not Found

```
{  
  "error": "Subtask not found"  
}
```